

“진화 2 (evolution2)” 문제 풀이

작성자: 구재현

부분문제 1

올바른 번호 배정은 유일하며, 최초 생명체에서 시작해서 유일한 자식을 반복해서 따라가면 이 번호 배정을 구할 수 있다.

부분문제 2

최초 생명체를 0번으로 배정하는 모든 배정이 올바르다. 즉, $N - 1$ 개의 수를 정렬하는 문제로 해석할 수 있다. 일반적인 Merge Sort를 구현하면 만점을 받을 수 있다.

부분문제 3

$A(v)$ 를, 생명체 v 를 루트로 한 서브트리의 정점들을 탄생 시점 기준으로 정렬한 수열이라고 하자. v 가 리프 노드인 경우, $A(v) = \{v\}$ 이다. v 가 리프 노드가 아닌 경우, v 의 두 자식 $\{w_1, w_2\}$ 에 대해 $A(w_1), A(w_2)$ 를 계산하고, Merge Sort의 요령으로 두 수열을 합쳐주면 만점을 받을 수 있다.

부분문제 4

정점 v 에 대해, $P(v)$ 를 가능한 모든 $A(v)$ 의 개수의 경우의 수라고 하자. v 의 자식을 $\{w_1, w_2, \dots, w_k\}$ 라고 했을 때, 다음과 같은 점화식이 성립한다:

$$P(v) = \begin{cases} 1 & \text{if } k = 0 \\ \binom{|A(w_1)| + \dots + |A(w_k)|}{|A(w_1)|, \dots, |A(w_k)|} \prod P(w_i) & \text{otherwise} \end{cases}$$

길이 L, S 의 두 수열을 $K \log_2 \binom{L+S}{L}$ 번 이하의 compare 연산으로 합칠 수 있는 알고리즘이 있다고 하자. $\binom{x_1+x_2+\dots+x_k}{x_1, x_2, \dots, x_k} = \binom{x_1+x_2}{x_2} \times \binom{x_1+x_2+x_3}{x_3} \times \dots \times \binom{x_1+x_2+\dots+x_k}{x_1, x_2, \dots, x_k}$ 이기 때문에, 이 알고리즘을 통해서 $A(w_1), A(w_2)$ 를 합쳐준 후 그 수열에 $A(w_3), A(w_4), \dots$ 를 순서대로 합쳐주면 전체 문제를 $K \log_2 P$ 번의 compare 연산으로 해결할 수 있다. 위와 같은 알고리즘이 있다면, 자식의 개수나 자식을 어떤 순서로 합쳐주는가는 전혀 중요하지 않고, 두 개의 수열을 적은 compare 연산으로 합쳐줄 수만 있으면 된다.

문제는 길이 L, S 의 두 수열을 최적의 연산으로 합칠 수 있는 알고리즘을 찾는 것으로 환원되었다. 일반성을 잃지 않고 $L \geq S$ 를 가정하자. 길이 L 의 수열을 큰 수열, 길이 S 의 수열을 작은 수열이라고 하자. 버킷을 만들듯이, 큰 수열을 S 개의 길이 $\lceil \frac{L}{S} \rceil$ 의 수열로 분할한다. 분할된 큰 수열의 맨 앞 원소를 모아서 작은 수열과 Merge해 주면, $2S - 1$ 번의 연산으로 각 작은 수열의 원소가 큰 수열의 어떠한 부분에 들어가는지를 알 수 있다. 해당 부분에서 어떤 특정한 위치에 들어가는지는 이분 탐색을 통해서 계산할 수 있다. 이를 사용하면 $2S + \log_2(\frac{L}{S} + 1)$ 번의 compare 연산으로 문제를 해결할 수 있고 80점 이상을 받을 수 있다.

위 풀이에서 적용할 수 있는 한 가지 최적화는, 각 분할된 수열의 크기가 정확히 $\frac{L}{S} + 1$ 일 필요가 없다는 것이다. 분할된 수열에서의 위치를 찾기 위해서 어차피 이분 탐색을 해야 하기 때문에, 분할된 수열의 크기를 2의 지수승으로 맞춰준 후 그에 따라서 분할의 개수를 줄이면 이분 탐색의 쿼리 횟수를 그대로 유지하면서 Merge Sort의 쿼리 횟수를 줄일 수 있다. 분할된 수열의 크기를 2^i 라고 두었을 때, 정확히 몇 번의 compare 연산이 필요한지를 간단한 수식으로 계산할 수 있다. 이 수식을 최소화하는 i 를 구한 후, 이 i 에 따라서 큰 수열을 분할하고 위 알고리즘을 적용하면 된다. 이 전략을 사용하면 100점을 받을 수 있다. 프로그램을 통해서 이 전략을 모든 $1 \leq S \leq L, S + L \leq 10000$ 에 대해서 검증해 보면, $K \leq 1.37$ 이 계산된다.

큰 K 가 나오는 경우는 대다수 S, L 이 굉장히 작기 때문에, 이 경우에 대해서 최적 전략을 하드 코딩하는 식으로 더 작은 K 를 얻을 수도 있다. 하지만 이는 만점을 받기 위해서 필요한 최적화는 아니다.

부분 점수를 받을 수 있는 다른 풀이 몇 가지를 소개한다: 작은 수열의 각 원소에서 이분 탐색을 하는데 특정 원소의 위치가 결정되면 다른 원소의 이분 탐색 범위를 결정된 위치로 한정시켜서 탐색 범위를 줄이는 풀이가 있다. 각 원소를 탐색하는 순서를 분할 정복처럼 중간 / 그 다음 중간과 같이 결정하면, 분할 정복 최적화와 비슷한 코드를 얻을 수 있으며, 이 방법도 $O(S \log_2 \frac{L}{S})$ 번의 쿼리를 사용함을 증명할 수 있다. 이 방법은 잘 최적화할 경우 만점을 얻을 수 있다. Treap을 사용하여 수열을 관리한 후, 이 글과 같은 방법으로 Treap을 합쳐주어도 $O(S \log_2 \frac{L}{S})$ 번의 쿼리를 사용한다. 이 방법은 60점 이상을 받는다.

Remark 1. Stirling's inequality에 의해:

$$(L + S) \log_2(L + S) - L \log_2(L) - S \log_2(S) = \log_2\left(\binom{L+S}{S}\right) + O(\log_2 L + \log_2 S - \log_2(L + S))$$

우변에 붙은 Error term인 $O(\log_2 L + \log_2 S - \log_2(L + S))$ 는 충분히 큰 L, S 에 대해 작다. (실제 증명에서 무시할 수 없지만) 이를 무시하고, $(L + S) \log_2(L + S) - L \log_2(L) - S \log_2(S) = \log_2\left(\binom{L+S}{S}\right)$ 라고 가정하자. 이 가정하에 $S + S \log(1 + \frac{L}{S})$ 번의 쿼리를 사용하는 알고리즘이 최적임을 증명한다. 모든 $x \in (0, 1]$ 에 대해 $x \leq \log_2(1 + x)$ 이다. 고로:

$$\begin{aligned} \frac{S}{L} &\leq \log_2\left(1 + \frac{S}{L}\right) \\ S &\leq L \log_2(S + L) - L \log_2(L) \\ S + S \log_2(S + L) &\leq (S + L) \log_2(S + L) - L \log_2(L) \\ S + S \log_2(S + L) - S \log_2(S) &\leq (S + L) \log_2(S + L) - L \log_2(L) - S \log_2(S) \\ S + S \log_2\left(1 + \frac{L}{S}\right) &\leq (S + L) \log_2(S + L) - L \log_2(L) - S \log_2(S) \end{aligned}$$

이와 같은 방식으로 $O(S + S \log(1 + \frac{L}{S}))$ 번의 쿼리를 사용하는 알고리즘이 asymptotically optimal함을 증명할 수 있다.

Remark 2. 입력이 Rooted tree가 아니라 DAG인 경우에도, $O(\log P)$ 번의 쿼리를 사용하여 숨겨진 순서를 복원하는 알고리즘이 존재한다. (Simpler Optimal Sorting from a Directed Acyclic Graph) 이 경우, P 는 DAG의 가능한 위상 정렬의 경우의 수에 대응된다. 트리에서 이 알고리즘을 적용할 경우, 최소 $K \geq 2$ 가 되기 때문에 만점을 받기는 어렵다.

“멋진 구간 (maxsum)” 문제 풀이

작성자: 구재현, 박상훈

부분문제 1

어떤 구간 $[l, r]$ 이 멋진 구간일 필요충분조건은 다음과 같다: $l \leq i \leq r$ 인 i 에 대해 $C[i] = B[i]$, 그렇지 않은 j 에 대해서는 $C[j] = A[j]$ 라고 할 때, $[l, r]$ 이 C 의 최대 부분배열이다. 배열 C 에서 Kadane's Algorithm을 적용하면 $O(N)$ 시간에 멋진 구간인지 판별할 수 있다. 따라서, 문제를 $O(N^3 + Q)$ 시간에 해결할 수 있다.

부분문제 2

배열 D 와 구간 $[l, r]$ 에 대해, $\text{info}_D(l, r) = (a, b, c, d)$ 를 다음과 같이 정의하자.

- a 는 $D[l] + \dots + D[i]$ 의 최댓값이다. (단, $l \leq i \leq r$) 즉, a 는 $D[l..r]$ 의 prefix sum 중 최댓값이다.
- b 는 $D[i] + \dots + D[r]$ 의 최댓값이다. (단, $l \leq i \leq r$) 즉, b 는 $D[l..r]$ 의 suffix sum 중 최댓값이다.
- c 는 $D[i] + \dots + D[j]$ 의 최댓값이다. (단, $l \leq i \leq j \leq r$) 즉, c 는 $D[l..r]$ 의 구간합 중 최댓값이다.
- d 는 $D[l] + \dots + D[r]$ 이다.

$\text{info}_D(l, m) = (a_1, b_1, c_1, d_1)$ 과 $\text{info}_D(m+1, r) = (a_2, b_2, c_2, d_2)$ 가 주어졌을 때, $\text{info}_D(l, r) = (a_3, b_3, c_3, d_3)$ 의 값은 다음과 같이 $O(1)$ 시간에 계산할 수 있다. 이러한 연산을 $+$ 를 사용하여 $\text{info}_D(l, m) + \text{info}_D(m+1, r)$ 과 같이 표기한다.

- $a_3 = \max(a_1, d_1 + a_2)$
- $b_3 = \max(b_2, d_2 + b_1)$
- $c_3 = \max(c_1, c_2, b_1 + a_2)$
- $d_3 = d_1 + d_2$

따라서, $\text{info}_A(i, j)$ 와 $\text{info}_B(i, j)$ 를 $O(N^2)$ 시간에 모두 계산할 수 있다. 구간 $[l, r]$ 이 멋진 구간인지 판별하려면 $\text{info}_A(0, l-1) + \text{info}_B(l, r) + \text{info}_A(r+1, N-1)$ 을 계산하면 충분하다. 따라서, 문제를 $O(N^2 + Q)$ 시간에 해결할 수 있다.

부분문제 4

info_A 와 info_B 를 세그먼트 트리에 저장하면 $O(\log N)$ 시간에 주어진 구간이 멋진 구간인지 확인할 수 있다. 따라서, 부분문제를 $O(N + Q \log N)$ 시간에 해결할 수 있다.

부분문제 3

다음 6가지 조건을 모두 만족하는 것은 구간 $[l, r]$ 이 멋진 구간인 것과 동치이다.

1. $A[i] + \dots + A[l-1] > 0$ 인 i 가 존재하지 않는다. ($0 \leq i \leq l-1$)

2. $A[r+1] + \dots + A[i] > 0$ 인 i 가 존재하지 않는다. ($r+1 \leq i \leq N-1$)
3. $B[l] + \dots + B[i] < 0$ 인 i 가 존재하지 않는다. ($l \leq i < r$)
4. $B[i] + \dots + B[r] < 0$ 인 i 가 존재하지 않는다. ($l < i \leq r$)
5. 어떤 i 가 존재하여, 임의의 x, y 에 대해 $A[x] + \dots + A[y] \leq B[l] \dots + B[i]$ 을 만족한다. ($l \leq i \leq r$, $r+1 \leq x \leq y \leq N-1$)
6. 어떤 i 가 존재하여, 임의의 x, y 에 대해 $A[x] + \dots + A[y] \leq B[i] \dots + B[r]$ 을 만족한다. ($l \leq i \leq r$, $0 \leq x \leq y \leq l-1$)

1번과 2번 조건은 각각 l, r 에만 의존하는 조건이다. $O(N)$ 시간에 모든 l 과 r 에 대해 이를 확인할 수 있다.

3번 조건의 경우, 어떤 함수 f_3 가 존재하여 $r \leq f_3(l)$ 일 때만 3번 조건을 만족한다. Sparse table과 이분탐색을 사용하면 $O(N \log N)$ 시간에 $f_3(0), \dots, f_3(N-1)$ 을 계산할 수 있다.

4번 조건의 경우, 어떤 함수 f_4 가 존재하여 $l \geq f_4(r)$ 일 때만 4번 조건을 만족한다. 3번 조건과 동일한 형태이므로 $O(N \log N)$ 시간에 $f_4(0), \dots, f_4(N-1)$ 을 계산할 수 있다.

5번 조건의 경우, l 을 고정하고 r 을 증가시키면 $A[x] + \dots + A[y]$ 의 최댓값은 감소하고, $B[l] + \dots + B[i]$ 의 최댓값은 증가하므로, 처음에는 조건을 만족하지 않다가 특정 지점부터 조건을 만족하기 시작한다. 따라서, 어떤 함수 f_5 가 존재하여 $r \geq f_5(l)$ 일 때만 5번 조건을 만족한다. Sparse table과 이분탐색을 사용하면 $O(N \log N)$ 시간에 $f_5(0), \dots, f_5(N-1)$ 을 계산할 수 있다.

6번 조건의 경우, 5번 조건과 동일한 형태이므로 어떤 함수 f_6 가 존재하여 $l \leq f_6(r)$ 일 때만 6번 조건을 만족한다. $O(N \log N)$ 시간에 $f_6(0), \dots, f_6(N-1)$ 을 계산할 수 있다.

1번 조건을 만족하는 l 에 대해, r 은 $f_5(l) \leq r \leq f_3(l)$ 을 만족해야 한다. 2번 조건을 만족하는 r 에 대해, l 은 $f_4(r) \leq l \leq f_6(r)$ 을 만족해야 한다. l 을 x 좌표, r 을 y 좌표에 대응시켜 2차원 좌표평면에 나타내면 x 축 또는 y 축과 평행한 선분 여러 개로 표현되며, 어떤 두 선분의 교점에 위치한 (l, r) 만 모든 조건을 만족한다.

따라서, 교점 개수를 세면 부분문제를 해결할 수 있다. 이는 세그먼트 트리와 스위핑을 사용하면 해결할 수 있다. 전체 시간복잡도는 $O((N+Q) \log N)$ 이다.

부분문제 5

직사각형 영역에 존재하는 교점 개수를 세는 쿼리를 처리하면 된다. x 좌표가 증가하는 방향으로 스위핑을 하면서 세그먼트 트리에 (현재 활성화된 x 축과 평행한 선분 개수, 현재까지의 답)을 저장하면 lazy propagation을 적용하여 정보를 관리할 수 있다. 쿼리의 답은 쿼리의 x 좌표 경계에서 오프라인으로 계산해줄 수 있다. 따라서, 문제를 $O((N+Q) \log N)$ 에 해결할 수 있다.

“넘버링 (numbering)” 문제 풀이

작성자: 구재현

어떠한 수열의 *비다양성* 을 $\sum_{0 \leq u, v \leq n-1} \mathbb{I}[A[u] = A[v]]$ 로 정의하자. (비다양성) + (다양성) + (다양성) = N^2 이기 때문에, 비다양성의 최솟값을 구하면 다양성의 최댓값을 쉽게 구할 수 있다. 아래 풀이에서는 비다양성을 최소화하는 문제를 해결한다.

또한, 굿 넘버링에 등장하는 서로 다른 숫자가 T 개일 때, $A[i] \in \{0, 1, \dots, T-1\}$ 임을 가정한다.

부분문제 4

문제의 정의에 따라서, $A[u] = A[v]$ 인 서로 다른 두 정점 u, v 를 잇는 경로에는 같은 숫자가 적혀 있어야 한다. 이에 따라, $A[u]$ 가 같은 정점들은 트리에서 서로 연결되어 있다. 이 정점들을 하나로 압축할 경우, 트리는 경로가 된다: 압축된 트리에서 차수 3 이상의 정점이 있다면, 이 정점보다 $A[v]$ 값이 작은 정점이 2 개 있거나 큰 정점이 2 개 있고, 그들간의 경로가 문제 조건을 만족하지 않는다.

최적해에서 $A[x] = 0, A[y] = T-1$ 인 두 정점을 고정하자. 위 관찰에 의해서, x 와 y 간의 경로에 속하지 않는 트리 간선 (u, v) 는 $A[u] = A[v]$ 를 항상 만족한다. x 와 y 간의 경로에 있는 간선들을 지우면, 트리는 몇 개의 컴포넌트로 분할되고, 각 컴포넌트에는 항상 같은 색이 배정되어야 한다. 이 경우, 경로를 따라가면서 각 컴포넌트마다 값이 증가하도록 다른 색을 배정해 주면 현재 조건에서 비다양성을 최소화하는 굿 넘버링을 얻을 수 있다. 고로, 이 경우 얻을 수 있는 비다양성의 최솟값은 각 컴포넌트 크기의 제곱의 합이다.

모든 가능한 x, y 에 대해서 위 절차를 구현해 주고, 비다양성의 최솟값을 계산해 주면 $O(n^3)$ 에 전체 문제를 해결할 수 있다.

부분문제 5

최적해에서 x, y 가 트리의 리프임을 가정할 수 있다. x 를 고정하고, 모든 y 에 대한 답을 DP로 해결하자. 다음과 같은 정의를 도입한다:

- $child[v]$: v 의 자식 집합
- $sub[v]$: v 의 서브트리 크기
- $DP[v]$: y 가 v 의 서브트리에 있다는 가정 하에 v 의 서브트리를 컴포넌트로 나눴을 때 비다양성의 최솟값

이 때, $DP[v] = \min_{w \in child[v]} (DP[w] + (sub[v] - sub[w])^2)$ 가 성립한다. x 를 고정했을 때 이 점화식은 $O(n)$ 시간에 계산할 수 있으니, 전체 문제는 $O(n^2)$ 시간에 해결할 수 있다.

부분문제 3

임의의 정점 (편의상 0 번 노드) 를 루트로 고정하고 DP를 계산하자. 정점 l 에 대해서, $LCA(x, y) = l$ 인 모든 (x, y) 에 대한 비다양성의 최솟값은, $\min_{u, v \in child[l], u \neq v} (DP[u] + DP[v] + (n - sub[u] - sub[v])^2)$ 와 같이 계산할 수 있다. 이를 모든 정점 l 에 대해서 계산해 주면, DP를 매번 다시 계산할 필요 없이 문제를 해결할 수 있다. 시간 복잡도는 모든 정점의 차수 합의 제곱으로, 이진 트리에서는 $O(n)$ 시간이지만 일반 트리에서는 여전히 $O(n^2)$ 이다.

부분문제 6

위 식을 풀어쓸 경우, $\min_{u < v, u, v \in \text{child}[l]} (DP[u] + DP[v] + (n - \text{sub}[u])^2 + \text{sub}[v]^2 - 2(n - \text{sub}[u])\text{sub}[v])$ 와 같이 전개된다. 고정된 v 에 대해서, 이는 일차 함수의 최솟값 쿼리와 동일하다. 필요한 일차 함수들을 기울기 순으로 정렬한 후 Convex Hull Trick을 사용하면, 전체 문제를 $O(n \log n)$ 에 해결할 수 있다. 출제진의 의도한 풀이는 이것이었으며, 실제 대부분의 학생들이 이렇게 해결하였다.

l 의 자식들을 $(\text{sub}[u], DP[u])$ 값을 기준으로 정렬하자. $\text{sub}[u]$ 가 같은 값이 3 개 이상 있다면, 이 중 $DP[u]$ 가 가장 작은 두 값을 제외하고는 모두 계산에서 무시해 주어도 무방하다. 그러한 값을 사용한다면 가장 작은 두 값으로 대체하는 것이 항상 이득이기 때문이다. 필요 없는 자식들을 제거한 후, 단순히 모든 자식 쌍을 순회할 경우 현재 입력 데이터에서 만점이 나온다. 코드가 짧기 때문에 대부분의 출제진들은 이렇게 해결하였다.

서로 다른 $\text{sub}[u]$ 값이 최대 $O(\sqrt{n})$ 개이기 때문에, 이 방법이 $O(n\sqrt{n})$ 의 시간 복잡도를 가지는 것은 증명할 수 있다. 하지만, 이보다 더 강한 증명은 알지 못한다. 출제진은 동적 계획법으로 모든 트리 중 (서로 다른 $\text{sub}[u]$ 값의 수)² 의 가능한 최대 합을 계산해 보았고, $n = 10000$ 에서 대략 5만, $n = 100000$ 에서 대략 60만 이하의 값이 계산되었다. 사용한 동적 계획법이 $O(n\sqrt{n})$ 의 메모리를 사용해서 $n = 1000000$ 에서의 최댓값을 산출하지는 못하였지만, 추세를 보았을 때는 의미 있는 반례가 존재하지 않을 것이라고 추측된다.

부분문제 9

어떠한 서로 다른 두 정점 u, v 에 대해서 u, v 를 모두 포함하는 사이클이 있다면, $A[u] = A[v]$ 여야 한다. 즉, 같은 이중 연결 요소에 속하는 노드들은 같은 값을 배정받아야 한다. 이중 연결 요소를 하나의 정점으로 합쳐주면, 그래프는 트리가 된다. 각 트리 정점이 원래 그래프의 여러 정점에 대응될 수 있다는 차이 외에는, 서브태스크 6과 동일한 풀이를 사용하여 해결할 수 있다. 이중 연결 요소는, 그래프의 절선을 구한 후, 절선이 아닌 간선들의 양 끝점을 (Union-Find 등으로) 합쳐주는 식으로 계산할 수 있다. 시간 복잡도는 (CHT 사용 기준) $O(m + n \log n)$ 이다.

“뗏목 제작 (raft)” 문제 풀이

작성자: 조승현

부분문제 1

주어진 KOI 숲과 IOI 숲의 나무를 모두 사용하여 뗏목을 만들 때, 높이가 h 인 직사각형의 가능한 최대 크기는 KOI 숲에서 높이 h 인 연속한 나무의 최대 갯수 $f_1(h)$ 와 IOI 숲에서 연속한 나무의 최대 갯수 $f_2(h)$ 에 대해 $h \times (f_1(h) + f_2(h))$ 가 된다. 여기서 h 는 나무들 중 적어도 하나의 높이와 일치해야 의미가 있으므로, 스택을 사용하여 히스토그램의 maximal한 직사각형을 구하는 방법을 이용하면 문제를 $O(N + M)$ 시간에 해결할 수 있다. 쿼리마다 이를 반복하면 $O(Q(N + M))$ 이 된다.

부분문제 2

위 방법으로 쿼리마다 $O(N + M)$ 시간에 문제를 해결하면 충분하다.

부분문제 4

높이가 50 이하이므로, 각 높이 h 에 대해 IOI 숲에서 높이가 h 인 연속한 구간은 겹치지 않는 구간들이다. 세그먼트 트리를 이용하면 각 높이 h 에 대한 최댓값을 $O(\log N)$ 시간에 구할 수 있으므로 $O(50 \times Q \log N)$ 시간에 문제를 해결할 수 있다.

이외의 부분문제를 해결하기 위해서는 다른 아이디어가 필요하다: KOI 숲에서 높이 h_1 , 너비 l_1 인 직사각형과 IOI 숲에서 높이 h_2 , 너비 l_2 인 직사각형이 존재한다면 이를 이용해 안정성이 $\min(h_1, h_2) \times (l_1 + l_2)$ 인 뗏목을 만들 수 있다. 또한, $(\infty, 0)$ 도 각 숲의 (h, l) 쌍에 포함시킨다면, 최대 안정성은 앞서 말한 경우를 통해서 나옴을 증명할 수 있다.

이제 다음과 같은 자료구조를 생각하자: (h_2, l_2) 이 주어졌을 때 KOI 숲에서 (h_1, l_1) 을 잘 선택하여 얻을 수 있는 $\min(h_1, h_2) \times (l_1 + l_2)$ 의 최댓값을 리턴

CHT와 세그먼트 트리를 이용하면 $O(N \log N)$ 시간의 초기화를 통해 $O(\log N)$ 시간에 (h_2, l_2) 쿼리를 처리하는 위 자료구조를 구현할 수 있다.

부분문제 3

IOI 숲에 해당하는 히스토그램에서 maximal한 직사각형을 모두 저장한 상태를 생각해보자. 문제 조건에 따라 이 직사각형들은 $[L[i], R[i]]$ 구간에 포함되거나, 완전히 포함하거나, 전혀 겹치지 않는다. 먼저 $[L[i], R[i]]$ 구간을 포함하는 경우는 (h_2, l_2) 쿼리 한번으로 해결할 수 있음을 알 수 있다. $[L[i], R[i]]$ 구간에 포함되는 경우는 모든 maximal한 직사각형 (h, l) 에 대해 앞서 말한 자료구조에 쿼리의 리턴값을 저장해두면 2D 레인지 쿼리가 되어 $O(M \log M)$ 또는 $O(M \log^2 M)$ 시간에 문제를 해결할 수 있다.

부분문제 9

부분문제 3에서 처리되지 않은 부분은 maximal한 직사각형 (l, r, h) 에 대해 $l \leq L[i] \leq r \leq R[i]$ 또는 $L[i] \leq l \leq R[i] \leq r$ 인 경우이다. 일반성을 잃지 않고 $L[i] \leq l \leq R[i] \leq r$ 라 하자. 각 구간별로 maximal한 직사각형 (l, r, h) 들을 저장하는 스택을 들고 있는 세그먼트 트리가 필요하다. maximal한 직사각형 (l, r, h) 에 대해 $[l, r]$ 에 해당하는 $O(\log M)$ 개 구간에 대해, 스택에 해당 직사각형을 추가하자. $(L[i], R[i])$ 쿼리에 대해 $x = R[i]$ 일 때 (l, r, h) 직사각형을 통해 얻을 수 있는 최대 안정성을 $f(x)$ 라 하자. 그러면 각 직사각형에 대해

f 의 그래프는 기울기가 증가하는 꺾은선 그래프이며, 한 x 에 대해 $f(x)$ 의 값은 앞서 만든 자료구조의 쿼리 한번으로 얻을 수 있다.

각 노드에 대해 스택에 있는 직사각형들의 $f(x)$ 의 maximum에 해당하는 그래프를 생각했을 때, 각 직사각형이 최대인 부분은 연속된 구간으로 표현된다. 각 교차하는 지점을 계산해놓을 수 있고, 이를 통해 세그먼트 트리를 $O(M \log^2 M \log N)$ 시간에 관리할 수 있다. 그리고 각 $(L[i], R[i])$ 쿼리는 $O(\log M)$ 개 노드에 대해 쿼리 한번씩이면 충분하므로 $O(Q \log^2 M)$ 시간이면 충분하다. 따라서, 문제를 $O(M \log^2 M \log N + Q \log M \log N)$ 시간에 해결할 수 있다.